

CSCI 3170 Tutorial Week 1

Introduction to Oracle and JDBC (I)

Kaiwen Chen (Kevin) SHB 904

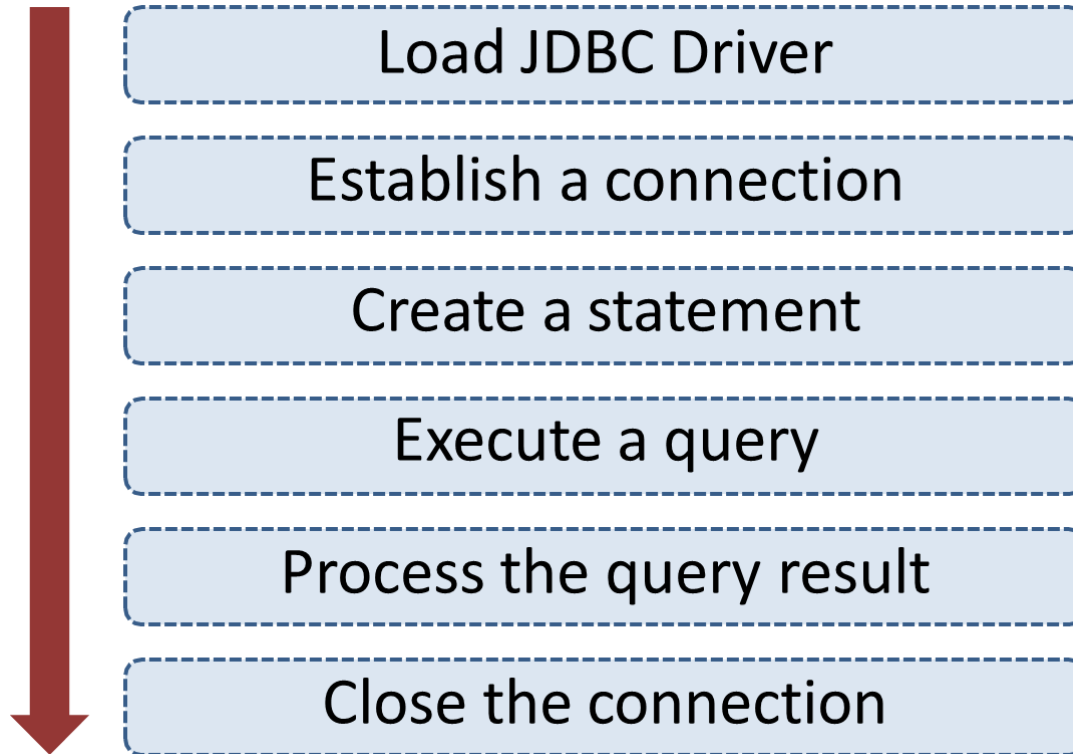
INTRO: ORACLE DBMS

- Oracle
 - Enterprise-grade
 - High reliability, security, and scalability
 - Full support for standard SQL and PL/SQL
 - Cross-platform support & rich APIs: Java (JDBC), C/C++, Python
 - ...
- Oracle is a relational database management system
 - Database consists of many tables (different entities and relations)
 - Row represents single entity (e.g., a customer)
 - Column represents attribute of entity (e.g., name, email, age)
 - Primary keys used to uniquely identify each row (and foreign keys to establish relationships between tables)

INTRO: JDBC

- Java Database Connectivity
 - Java API for accessing different kind of data
- Manage three activities
 - Connect to a database
 - Send queries to the database
 - Retrieve results from the database

Core Workflow of JDBC



On Obtaining Oracle JDBC Driver!

- Link:
<https://www.oracle.com/database/technologies/appdev/jdbc-downloads.html>
- Oracle Database 21c:
ojdbc17.jar (Supports JDK 26,
connects to Oracle 12.2)

Oracle Database JDBC Driver

Official Downloads & Setup for CSCI 3170 (Lab 1)

1. Required Driver Version: Oracle Database 21c

Driver File: **ojdbc17.jar**

- Supports JDK 26 (as required for CSCI 3170)
- Fully compatible with departmental Oracle 12.2 server

• Download from Course Website or Oracle JDBC Downloads page:
<https://www.oracle.com/database/technologies/appdev/jdbc-downloads.html>

• **Note:** Do not unzip the JAR file. Save it in a stable course folder.

2. Adding ojdbc17.jar to IntelliJ IDEA

Step 1 In IntelliJ IDEA, go to File -> Project Structure (Cmd+; or Ctrl+Alt+Shift+S).

Step 2 Select Libraries in the left pane, click the + button, and select Java.

Step 3 Browse and select the downloaded ojdbc17.jar file.

Step 4 When prompted, attach the library to CSCI3170-Labs module.

Step 5 Click Apply, then OK. The driver is now under External Libraries!

Merely downloading the JAR is not enough; it must be added as a dependency.

Load Oracle JDBC Driver

- Use the Oracle 21c **ojdbc17.jar** (see prev slide)
 - Add to IntelliJ Project Structure or via `-classpath`

```
java -classpath ./ojdbc17.jar:. <class_file>
```

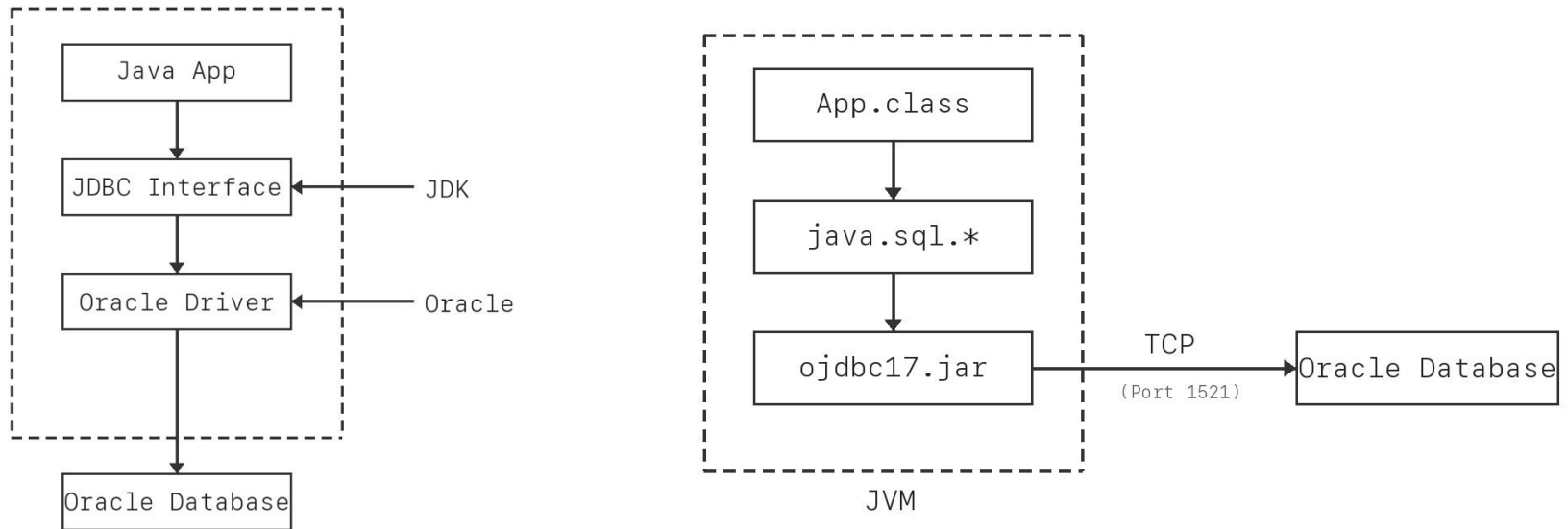
^ IntelliJ / Command line dependency

- Importing required packages for JDBC API

```
import java.sql.*;
```

^ Java source code import

JDBC Architecture with Oracle



Establish a Connection

- Load JDBC Driver for Oracle DBMS

```
try {  
    Class<?> driver = Class.forName("oracle.jdbc.OracleDriver");  
} catch (Exception x) {  
    System.err.println("Unable to load the driver class!");  
}
```

- Establish a Connection

```
Connection conn =  
DriverManager.getConnection("jdbc:oracle:thin:@db18.cse.cuhk.edu.hk:15  
21:db18", "username", "password");
```

Create and execute a statement

- Create a statement object

```
Statement stmt = conn.createStatement();
```

- Execute a statement using the object

```
Stmt.executeUpdate("CREATE TABLE Student " +  
"(UserID VARCHAR2(10), " +  
"Password VARCHAR(8))");
```

- Retrieve data from result set

```
ResultSet rs = stmt.executeQuery("SELECT * FROM temp");
```

- Process the query result

```
String user_id;  
String password;  
  
while (rs.next()){  
    user_id = rs.getString(1);  
    password = rs.getString(2);  
}
```

rs.next() moves the cursor
down one row from its
current position

JDBC Datatypes

NUMBER /
VARCHAR2
(Oracle SQL
Types)



JDBC Type	Java Type	Method
INT	int	getInt
REAL	float	getFloat
FLOAT	double	getDouble
DOUBLE	double	getDouble
CHAR	String	getString
VARCHAR	String	getString
DATE	java.sql.Date	getDate
TIMESTAMP	TIMESTAMP	getTimeStamp

PreparedStatement

- Prepared Statements
 - For when handling a large number of records

```
PreparedStatement pstmt = con.prepareStatement( "INSERT INTO  
Student VALUES (?, ?)");  
  
for (int i = 0, i < student.length, i++){  
    pstmt.setString(1, student[i][0]);  
    pstmt.setString(2, student[i][1]);  
    pstmt.executeUpdate();  
}
```

- Gives a better performance
 - SQL statement only needs to be compiled once
- More secure
 - can prevent some (but not all) SQL injections

Closing the JDBC connection

- Avoid holding unnecessary recourses

```
/* destroy the result set object */  
rs.close();  
  
/* destroy the statement object */  
stmt.close() ;  
  
/* destroy the prepared statement object */  
pstmt.close();  
  
/* destroy the connection */  
conn.close();
```

JDBC Connection Pool

- When performing JDBC operations, if each operation comes with an open connection, operation, and close connection, then the overhead of creating and destroying JDBC connections is too great!
- Reuse already created connections via Connection Pools

```
HikariConfig config = new HikariConfig();
config.setJdbcUrl("jdbc:oracle:thin:@db18.cse.cuhk.edu.hk:1521:db18");
config.setUsername("username");
config.setPassword("password");
config.addDataSourceProperty("connectionTimeout", "1000");
config.addDataSourceProperty("idleTimeout", "60000");
config.addDataSourceProperty("maximumPoolSize", "10");
DataSource ds = new HikariDataSource(config);

try (Connection conn = ds.getConnection()) {
    ...
}
```

Thank you !